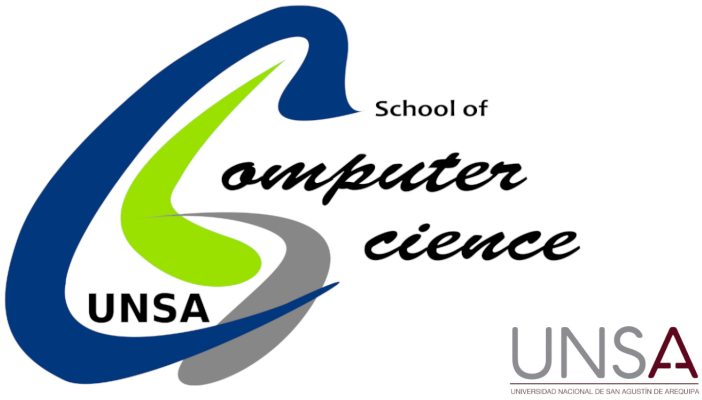
UNIVERSIDAD NACIONAL DE SAN AGUSTÍN
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS



ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN
COMPUTACION PARALELA Y DISTRIBUIDA

RÉPLICA EXPERIMENTAL Y EVALUACIÓN DE RENDIMIENTO DE LLVM MPI PROXY PLUGIN (MPP) FRENTE A MPI+OPENMP EN UN CLÚSTER HETEROGÉNEO COMPACTO

INTEGRANTES:

- Lopez Zegarra, Ivan Alexander

DOCENTE:

ALVARO HENRY MAMANI ALIAGA

AREQUIPA – PERÚ
2026

Índice

1. Resumen Ejecutivo 1
2. Introducción y Motivación 2
   2.1. El Desafío de la Escalabilidad Trans-Nodo en OpenMP 2
   2.2. El Modelo Híbrido (MPI + OpenMP) y sus Limitaciones 2
   2.3. La Propuesta de LLVM: MPI Proxy Plugin (MPP) 2
3. Fundamentos Teóricos y Arquitectura de Software 3
   3.1. Pila Modular de Offloading en LLVM 3
   3.2. Principio de Funcionamiento de MPP 3
   3.3. Comparación Conceptual: MPP frente a MPI + OpenMP 4
4. Entorno Experimental y Banco de Pruebas 4
   4.1. Comparación Arquitectural: Santos Dumont 2 vs Clúster P3 Tiny 4
   4.2. Infraestructura del Clúster Físico 5
   4.3. El Benchmark Task Bench y Grafo Stencil 1D 6
   4.4. Métricas Formales de Evaluación 6
5. Ingeniería del Entorno: Diagnóstico y Solución de Fallos Críticos 7
   5.1. Fallo 1: Bloqueo Indefinido de Proxies por Doble Registro (`PluginManager.h`) 7
   5.2. Fallo 2: Incompatibilidad Crítica de MPICH en Ubuntu 24.04 (Launchpad #2072338) 8
   5.3. Fallo 3: Interbloqueo Estático en Modo Híbrido (`tb_mpi`) 8
   5.4. Fallo 4: Segfault en Destructor de EventSystem 9
6. Resultados Experimentales y Análisis Comparativo 9
   6.1. Rendimiento Pico Mononodo FP32 en RTX A1000 9
   6.2. Análisis de Granularidad y Curvas de Eficiencia 9
   6.3. Comparativa del Umbral METG(50%) 10
      6.3.1. Interpretación Física de los Resultados: 11
   6.4. Escalabilidad Débil (Weak Scaling de 1 a 10 Nodos) 11
7. Discusión Técnica y Compensaciones (Trade-Offs) 12
   7.1. Análisis de Productividad vs Sobrecosto de Ejecución 12
   7.2. Guía de Decisión Arquitectural 13
8. Conclusiones y Trabajo Futuro 13
   8.1. Conclusiones de la Réplica 13
   8.2. Líneas de Trabajo Futuro 14

Bibliografía 14

1. Resumen Ejecutivo

El presente informe documenta la réplica experimental, el análisis exhaustivo de rendimiento y la resolución de incidencias de ingeniería de bajo nivel en torno al artículo científico **«Can OpenMP Scale Beyond the Node? A Performance Evaluation of Remote Offloading via the MPI Proxy Plugin»** [1], presentado en el *Simpósio em Sistemas Computacionais de Alto Desempenho* (SSCAD 2025).

La investigación original evaluó el mecanismo **MPI Proxy Plugin (MPP)** integrado en el runtime de **LLVM/OpenMP**, el cual permite orquestar aceleradores (GPUs) distribuidos físicamente a través de una red de interconexión sin alterar las directivas estándares `#pragma omp target`. Aquellos experimentos fueron conducidos sobre el supercomputador **Santos Dumont 2** (LNCC, Brasil) con nodos equipados con aceleradores masivos para centros de datos (**NVIDIA H100 SXM** de 80 GB y **AMD Instinct MI300A APU** de 128 GB) interconectados mediante **InfiniBand NDR (400 Gbps)** con soporte GPU-Aware y RDMA.

En esta réplica, trasladamos y evaluamos dicho paradigma en un clúster físico compacto de tipo **Mini-PC / Workstation Edge HPC**, constituido por 10 nodos **Lenovo ThinkStation P3 Tiny (Gen 2)** con procesadores **Intel Core Ultra 9 285K/T (Arrow Lake)**, aceleradores discretos **NVIDIA RTX A1000 (4 GB GDDR6)** e interconexión de red estándar **Ethernet Gigabit (1 Gbps)** sobre **OpenMPI 4.1.6** y **Clang 21.0**.

A través del framework parametrizado **Task Bench** [2] sobre un patrón de dependencia tipo *Stencil 1D (Radix 5)*, verificamos empíricamente:

1. **Viabilidad funcional del offloading remoto transparente:** Es factible desplegar y ejecutar aplicaciones OpenMP multi-GPU distribuidas sobre 10 nodos físicos desde un único proceso máster sin programar intercambios de mensajes explícitos en el código de usuario.
2. **Granularidad Mínima Efectiva (METG al 50% de eficiencia):** La brecha de granularidad se ensancha marcadamente al pasar de redes de ultra-baja latencia (InfiniBand) a redes commodity (Ethernet). Mientras que el modelo híbrido tradicional **MPI+OpenMP** retiene un $\mathrm{METG}(50\%)$ sub-milisegundo (0.65 ms en RTX A1000), el modelo **MPP** requiere tareas de granularidad media o gruesa (7.88 ms) para amortizar el sobrecosto de ida y vuelta (*roundtrip*) de la capa proxy sobre sockets TCP.
3. **Escalabilidad Débil y Saturación Computacional:** Para tareas intensivas de cómputo en coma flotante ($\geq$ 10 ms), MPP escala de forma lineal prácticamente perfecta, alcanzando 41.5 $\mathrm{TFLOP/s}$ acumulados en FP32 sobre las 10 GPUs discretas del clúster (frente a 4.42 $\mathrm{TFLOP/s}$ en un único nodo).
4. **Resolución de Bugs Críticos:** Se diagnosticaron y parcharon exitosamente cuatro anomalías severas del software base: un bloqueo de finalización en `PluginManager.h` (LLVM) por doble registro de librerías, un fallo del paquete MPICH en Ubuntu 24.04 (Launchpad #2072338) solventado mediante OpenMPI y aislamiento MCA, un interbloqueo estático por sondeo ciego de proxies en modo híbrido resuelto con la variable `OMPTARGET_DISABLE_MPI_PLUGIN`, y una protección contra segfault en el destructor de `EventSystem`.

Palabras clave: Computación Paralela y Distribuida, OpenMP Remote Offloading, LLVM liboffload, MPI Proxy Plugin, Task Bench, METG, NVIDIA RTX A1000, Clúster Heterogéneo.

—

2. Introducción y Motivación

2.1. El Desafío de la Escalabilidad Trans-Nodo en OpenMP

El estándar **OpenMP** [3] se ha consolidado a nivel mundial como la interfaz de programación de aplicaciones (API) dominante para la explotación de paralelismo en arquitecturas de memoria compartida. Con la introducción y maduración de la directiva `target` (a partir de OpenMP 4.0 y ampliada en 5.x) [4], los desarrolladores pueden transferir kernels computacionalmente pesados a coprocesadores y aceleradores gráficos (GPUs) mediante directivas declarativas sencillas como `#pragma omp target teams distribute parallel for`.

Sin embargo, históricamente OpenMP ha estado restringido estrictamente a las fronteras del nodo físico local. Todo el modelo de ejecución asume que el host y los aceleradores residen bajo el control de un único sistema operativo con acceso a memoria compartida o enlaces locales de alta velocidad como PCIe o NVLink [5]. Cuando una simulación científica o un pipeline de aprendizaje automático excede la capacidad de memoria o la potencia de cálculo de un solo servidor, el programador se ve obligado a reescribir sustancialmente su código adoptando un **modelo híbrido**, usualmente combinando **MPI** (*Message Passing Interface*) entre nodos y **OpenMP** dentro de cada nodo.

2.2. El Modelo Híbrido (MPI + OpenMP) y sus Limitaciones

El enfoque tradicional **MPI + OpenMP** requiere un paradigma de computación SPMD (*Single Program, Multiple Data*) [6]. Cada nodo ejecuta una o más instancias independientes del proceso, y el programador debe asumir manualmente la carga cognitiva de:

- Descomponer geométricamente el dominio de datos en particiones disjuntas.
- Identificar y sincronizar celdas fantasma (*ghost cells* o halos) mediante llamadas complejas a `MPI_Isend`, `MPI_Irecv`, `MPI_Waitall` o comunicaciones colectivas.
- Sincronizar el avance temporal del cómputo GPU con las transferencias de red, evitando condiciones de carrera e interbloqueos (*deadlocks*).

Esta dualidad fragmenta la semántica del programa: la concurrencia local se gestiona con directivas declarativas de alto nivel, mientras que la distribución inter-nodo se resuelve mediante llamadas imperativas de bajo nivel con control manual de búferes y rangos.

2.3. La Propuesta de LLVM: MPI Proxy Plugin (MPP)

Para superar esta dicotomía, investigadores de la Universidade Estadual de Campinas (UNICAMP) concibieron el **MPI Proxy Plugin (MPP)** [1]. MPP extiende la infraestructura modular de offloading del compilador LLVM (`liboffload` / `libomptarget`) para interceptar las regiones `#pragma omp target` dirigidas a dispositivos lógicos remotos y canalizarlas de manera transparente a través de un backend de red basado en MPI.

Bajo MPP:

- La aplicación se compila y ejecuta como un **único proceso host OpenMP centralizado**.
- Los aceleradores situados en nodos remotos son enumerados como dispositivos adicionales del sistema (`device(0)`, `device(1)`, ..., `device(N-1)`).
- Una capa de daemons proxies (`llvm-offload-mpi-proxy-device`) ejecutándose en cada nodo worker recibe las órdenes de ejecución y los datos correspondientes, invocando el backend local del fabricante (CUDA o ROCm) y devolviendo los resultados al host orquestador.

El objetivo central de la réplica abordada en este informe es constatar empíricamente si esta atractiva simplificación del modelo de programación mantiene la competitividad de rendimiento y escalabilidad cuando se traslada desde los supercomputadores de clase mundial hasta un clúster físico compacto de estación de trabajo con tarjetas gráficas de gama de entrada y red Ethernet comercial.

—

3. Fundamentos Teóricos y Arquitectura de Software

3.1. Pila Modular de Offloading en LLVM

La implementación de OpenMP en el compilador LLVM se sustenta en una arquitectura altamente modular compuesta por dos bibliotecas fundamentales [4], [7]:

1. **`libomp`:** Runtime clásico encargado de la gestión de hilos de CPU, creación de regiones paralelas (`fork-join`), bucles compartidos y sincronización basada en barreras o cerrojos.
2. **`liboffload` (anteriormente `libomptarget`):** Capa de abstracción que intermedia entre las directivas `target` y los aceleradores de hardware. `liboffload` define una interfaz independiente del dispositivo y carga dinámicamente complementos específicos para cada arquitectura (denominados *plugins NextGen*).

Componente LLVM	Plugin / Módulo	Responsabilidad Técnica
`libomp`	Host Runtime	Planificación de hilos de CPU, directivas `omp task`, `barrier`.
`liboffload`	Abstracción Base	Gestión de tablas de mapeo de punteros host--dispositivo y colas de tareas asíncronas.
`omptarget.rtl.cuda`	Plugin CUDA	Interacción con el CUDA Driver API (`cuLaunchKernel`, `cuMemcpyDtoHAsync`).
`omptarget.rtl.amdgpu`	Plugin ROCm	Interacción con el HSA / HIP Driver API para aceleradores AMD CDNA.
`omptarget.rtl.mpi`	**MPI Proxy Plugin**	**Intercepta invocaciones de target remoto y las delega mediante MPI.**

Tabla 1: Arquitectura de capas del runtime de offloading en LLVM/OpenMP.

3.2. Principio de Funcionamiento de MPP

Cuando el código de usuario ejecuta una directiva con cláusula de dispositivo dirigida a un nodo remoto:

```
#pragma omp target teams distribute parallel for device(k) map(to: in[:N]) map(from:
out[:N])
for (int i = 0; i < N; i++) {
    out[i] = compute_kernel(in[i]);
}
```

Si el identificador de dispositivo `k` corresponde a una GPU remota, el plugin `omptarget.rtl.mpi` en el nodo host:

1. Empaqueta los metadatos de lanzamiento (número de bloques, límite de hilos, dirección y tamaño de los punteros mapeados).
2. Transmite un mensaje de control al rango MPI asociado mediante primitivas no bloqueantes (`MPI_Isend`).

3. Transfiere los búferes de memoria asociados mediante canales MPI punto a punto.
4. En el nodo destino, el daemon worker `llvm-offload-mpi-proxy-device` desempaqueta los datos, reserva memoria en la GPU local a través del plugin nativo del dispositivo (`omptarget.rtl.cuda`), despacha el kernel a la GPU y aguarda la señal de finalización.
5. Una vez completado, el proxy transmite los datos mapeados con `map(from:)` y la señal de término de vuelta al proceso orquestador.

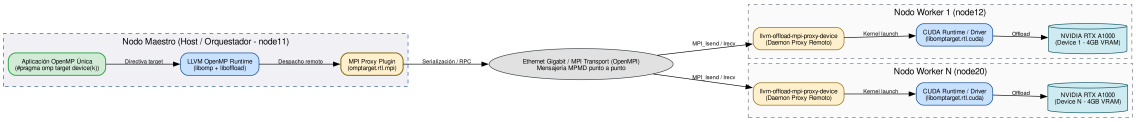


Figura 1: Arquitectura de ejecución de LLVM MPP frente al modelo híbrido tradicional. En MPP, un único proceso orquestador despacha tareas a proxies remotos sin requerir particionado SPMD manual.

3.3. Comparación Conceptual: MPP frente a MPI + OpenMP

En la Tabla 2 se resumen las diferencias paradigmáticas entre ambas aproximaciones.

Criterio de Evaluación	**Modelo Tradicional (MPI + OpenMP)**	**MPI Proxy Plugin (LLVM MPP)**
Modelo de Ejecución	SPMD distribuido (múltiples procesos independientes en red).	Host-Centric centralizado (un proceso máster, daemons pasivos).
Gestión de Datos	Particionado manual de arreglos y manejo explícito de celdas fantasma.	Mapeo implícito o directo mediante directivas `map(to/from)` de OpenMP.
Complejidad de Código	Alta: cientos de líneas de lógica `MPI_Send/Recv`, empaquetado y cálculo de offsets.	Mínima: código OpenMP idéntico al de un sistema mononodo multi-GPU.
Sobrecosto de Despacho	Bajo: comunicación directa punto a punto entre rangos adyacentes.	Moderado a Alto: serialización de metadatos de offloading y saltos por la capa proxy.
Cuello de Botella	Contención en la red inter-nodo por sincronizaciones globales.	Saturación del proceso host al centralizar el grafo de dependencias y el despacho.

Tabla 2: Comparación de características operativas entre MPI+OpenMP y LLVM MPP.

—

4. Entorno Experimental y Banco de Pruebas

4.1. Comparación Arquitectural: Santos Dumont 2 vs Clúster P3 Tiny

Para comprender el alcance y los desafíos de la réplica, es imperativo contrastar el hardware original empleado en la publicación de SSCAD 2025 con la plataforma experimental de laboratorio disponible para este estudio.

Característica	**Supercomputador Santos Dumont 2 (Paper)**	**Clúster Replicado (Este Estudio)**
Formato y Nodos	2 nodos clase Supercomputador (LNCC)	**10 nodos Lenovo ThinkStation P3 Tiny (Gen 2)**
Procesador Central	Dual Intel Xeon SHR M9468 / AMD Zen 4	**Intel Core Ultra 9 285K/T (24C / 24T Arrow Lake)**
Memoria RAM Host	1024 GB (sd2-h100) / 356 GB (sd2--mi300a)	**32 GB DDR5 por nodo (320 GB en el clúster)**
Aceleradores GPU	NVIDIA H100 SXM (80 GB) / AMD MI300A (128 GB)	**10x NVIDIA RTX A1000 Laptop GPU (4 GB GDDR6)**
Arquitectura GPU	Hopper (sm_90) / CDNA 3 (gfx942)	**Ampere (sm_86, GA107, 16 SMs, 2048 Cores)**
TDP del Acelerador	700 W (H100) / 550 W (MI300A)	**35 W por GPU (Bajo consumo / Formato ultra-compacto)**
Interconexión de Red	**InfiniBand NDR (400 Gbps, RDMA, sub-mus)**	**Gigabit Ethernet (1 Gbps, sockets TCP, 0.1-1 ms)**
Rendimiento FP32 Pico	60 TFLOP/s (H100) / 45 TFLOP/s (MI300A)	**4.42 TFLOP/s medidos por nodo (44.2 TFLOP/s total)**
Pila de Compilación	Clang 21.0, MPICH 4.3, UCX 1.18, CUDA 12.8	**Clang 21.0 (MPP patched), OpenMPI 4.1.6, CUDA 12.0**

Tabla 3: Comparación exhaustiva de hardware entre el paper original y el clúster replicado.

Como se desprende de la Tabla 3, la réplica impone dos diferencias arquitecturales drásticas:

1. **Capacidad de Cómputo por Nodo:** La GPU RTX A1000 ofrece aproximadamente un orden de magnitud menor rendimiento en FP32 y memoria VRAM (4 GB vs 80 GB), lo que exige adaptar cuidadosamente los tamaños de problema para evitar desbordamientos de memoria.
2. **Régimen de Red (InfiniBand vs Ethernet):** La red Ethernet de 1 Gbps introduce una latencia física y de pila de protocolos sensiblemente mayor que InfiniBand con UCX y GPU-Direct RDMA. Esta condición resulta sumamente enriquecedora para la investigación, pues permite evaluar la robustez y los límites de amortización de MPP bajo condiciones de red restrictivas (entornos comunes en laboratorios académicos o clústeres de borde).

4.2. Infraestructura del Clúster Físico

El clúster está compuesto por 10 estaciones de trabajo ultracompactas Lenovo ThinkStation P3 Tiny interconectadas a un conmutador de red de área local:

- **Nodo Maestro / Orquestador:** `node11` (`alumno11`), desde donde se lanza la orquestación, se compilan los binarios y se administra el almacenamiento.
- **Nodos Esclavos / Workers:** `node12` a `node20` (`alumno12` a `alumno20`), que actúan como destinos de ejecución remota tanto para los daemons `llvm-offload-mpi-proxy-device` de MPP como para los rangos MPI subordinados.

La sincronización de binarios y bibliotecas en `/opt/llvm-mpp` se automatizó mediante scripts de despliegue basados en SSH sin contraseña (`scripts/deploy_cluster.sh`), garantizando coherencia binaria exacta en los 10 nodos.

4.3. El Benchmark Task Bench y Grafo Stencil 1D

Para aislar de manera reproducible los sobrecostos del runtime sin la interferencia de lógicas de aplicación arbitrarias, se empleó **Task Bench** [2], un framework de evaluación de modelos de programación paralela ampliamente adoptado en la comunidad HPC.

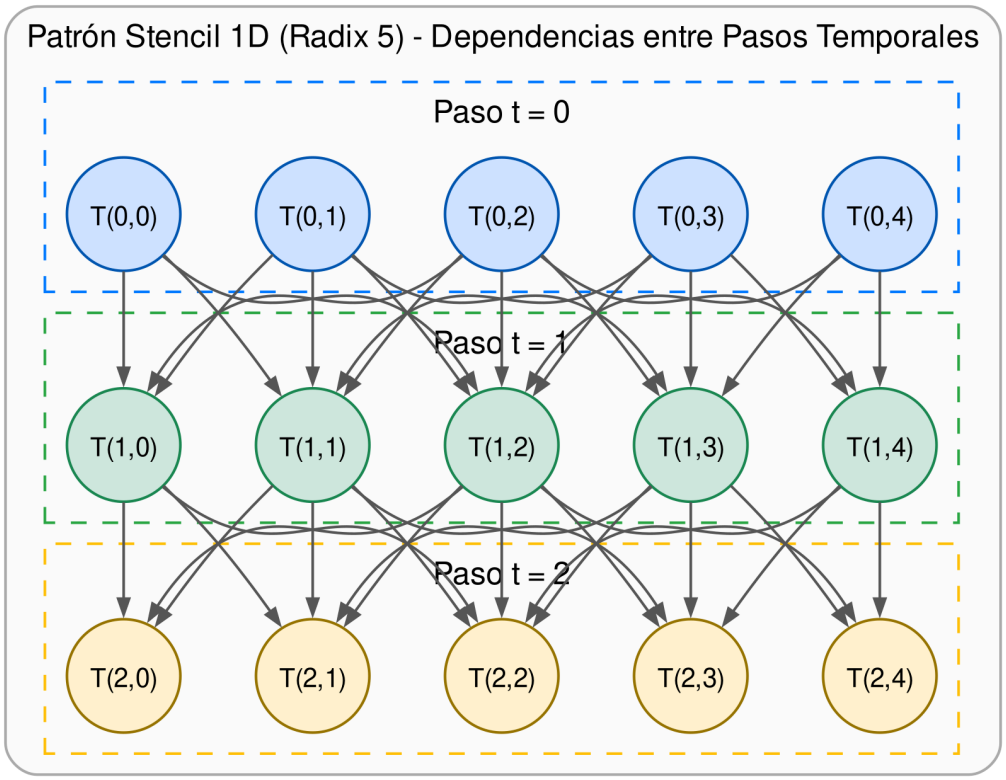


Figura 2: Estructura del grafo de tareas Stencil 1D (Radix 5) evaluado. En cada paso temporal t, una tarea depende de los valores computados por sus 5 celdas vecinas del paso temporal anterior $t-1$.

Las configuraciones clave del benchmark abarcaron:

- **Patrón de Dependencia:** Grafo `stencil_1d` con `radix = 5`. Como se ilustra en la Figura 2, cada tarea de la fila t depende de cinco tareas adyacentes de la fila $t-1$.
- **Kernel Computacional (`compute_bound_gpu`):** Diseñado específicamente por los autores del paper para someter a estrés las unidades de cómputo en coma flotante (FP32) de las GPUs. Consiste en operaciones FMA (*Fused Multiply-Add*) desenrolladas 16 veces (`#pragma unroll 16`).
- **Cálculo de Operaciones en Coma Flotante:** Con `nblocks = 1024` y `tpblock = 512` (es decir, 524, 288 hilos concurrentes por kernel):

$$\text{Total FLOPs} = \text{nblocks} \times \text{tpblock} \times \text{iterations} \times 32$$

- **Variación de Carga:** El parámetro `size` modula el número de iteraciones del kernel ($\text{iter} = 2^{\{20-\text{ size}\}}$), abarcando desde tareas casi instantáneas (`size = 18`, pocas microsegundos de cálculo) hasta tareas masivas (`size = 2`, 500 ms a 1 s de ejecución por tarea).

4.4. Métricas Formales de Evaluación

Para cuantificar el comportamiento del sistema se computaron rigurosamente las siguientes métricas:

1. **Granularidad de Tarea (τ):** Expresa la duración promedio del cómputo útil ejecutado por cada tarea en un dispositivo individual:

$$\tau = \frac{t_{\text{wall}} \times N_{\text{devices}} \times 10^3}{N_{\text{tasks}}} \quad [\text{ms}]$$

   donde t_{wall} es el tiempo total transcurrido (*Wall Time*), N_{devices} el número de GPUs y N_{tasks} el número total de tareas del grafo.
2. **Rendimiento Efectivo Sostenido:**

$$\text{TFLOP/s} = \frac{\text{Total FLOPs} \times 10^{-12}}{t_{\text{wall}}}$$

3. **Eficiencia Relativa de Cómputo (η):**

$$\eta = \frac{\text{TFLOP/s}}{\text{TFLOP/s}_{\text{máximo}}} \times 100\%$$

4. **Minimum Effective Task Granularity ($\text{METG}(50\%)$):** Representa el valor crítico de granularidad de tarea (τ) en el cual el sistema sostiene al menos el 50% de su rendimiento pico. Cuanto menor sea el METG, mayor es la capacidad del runtime para procesar paralelismo de grano fino sin sucumbir a las sobrecargas de coordinación.
5. **Escalabilidad Débil (*Weak Scaling*):** Se incrementa el tamaño del grafo en proporción directa al número de dispositivos (width $= N_{\text{devices}}$, manteniendo fijas 20 tareas por GPU), evaluando si el tiempo de ejecución permanece constante.

—

5. Ingeniería del Entorno: Diagnóstico y Solución de Fallos Críticos

Uno de los aportes más significativos de esta réplica fue la depuración y resolución de cuatro anomalías severas que impedían la ejecución en el clúster de pruebas. Estos fallos radicaban en incompatibilidades entre la distribución base del sistema operativo (Ubuntu 24.04), la biblioteca MPI y fallos de gestión de memoria en el código fuente de LLVM liboffload.

5.1. Fallo 1: Bloqueo Indefinido de Proxies por Doble Registro (`PluginManager.h`)

- **Síntoma:** Al ejecutar aplicaciones mediante MPP (`tb_mpp`), el cómputo finalizaba correctamente en las GPUs remotas, pero el proceso principal nunca retornaba al prompt de Linux, quedando colgado indefinidamente. La inspección mediante `gdb` y `strace` reveló que los procesos proxy remotos `llvm-offload-mpi-proxy-device` permanecían bloqueados en una llamada bloqueante `MPI_Recv`.
- **Análisis de Causa Raíz:** En el código fuente de LLVM (`offload/include/PluginManager.h`), el método `registerDelayedLibraries` invocaba la función de alto nivel `__tgt_register_lib(Desc)` para imágenes binarias diferidas. Esta función invocaba internamente a `registerLib`, provocando que el contador de referencias atómico (`RefCount`) de la librería de offloading se incrementara dos veces (`RefCount = 2`). Al finalizar la aplicación, la rutina `deinitRuntime` verificaba estrictamente la condición:

```
if (RefCount == 1) {
    PM->deinit(); // Envía mensaje de apagado MPI a los proxies
}
```

Al ser `RefCount = 2`, el decremento final lo dejaba en 1 en lugar de 0, impidiendo invocar `PM->deinit()`. En consecuencia, la orden de terminación jamás era despachada por MPI, y los proxies remotos aguardaban indefinidamente.

- **Solución Implementada:** Se reemplazó la llamada anidada por `registerLib(Desc)` directamente:

```
--- a/offload/include/PluginManager.h
+++ b/offload/include/PluginManager.h
@@ -154,7 +154,7 @@ struct PluginManager {
       for (auto &Desc : DelayedImages)
-        __tgt_register_lib(Desc);
+        registerLib(Desc);
       DelayedImages.clear();
```

Con este parche (`patches/llvm_mpp_fixes.patch`), el contador de referencias decrementa con precisión a cero al terminar, y los proxies finalizan limpiamente con código de salida 0.

5.2. Fallo 2: Incompatibilidad Crítica de MPICH en Ubuntu 24.04 (Launchpad #2072338)

- **Síntoma:** Al intentar lanzar el benchmark bajo el orquestador MPICH empaquetado en Ubuntu 24.04, las ejecuciones MPMD (`llvm-offload-mpi-proxy-device : tb_mpp`) fallaban catastróficamente al inicializar.
- **Análisis de Causa Raíz:** Se descubrió un bug crítico reportado en los repositorios oficiales de Ubuntu (Launchpad Bug #2072338). Bajo esquemas MPMD con múltiples ejecutables en la misma línea de comandos, la implementación de MPICH 4.2 empaquetada retorna erróneamente `rank = 0` y `size = 1` en todos los procesos invocados, imposibilitando la asignación de roles maestro-esclavo.
- **Solución Implementada:** Se migró la totalidad de la infraestructura de red a **OpenMPI 4.1.6**. Para asegurar un rendimiento predecible y evitar que OpenMPI intentara comunicarse a través de interfaces de red virtuales (Docker, bridges locales o túneles VPN), se configuró de forma mandatoria en `openmpi-mca-params.conf`:

```
btl_tcp_if_include = wlp129s0f0
oob_tcp_if_include = wlp129s0f0
```

garantizando que todo el tráfico de datos y de control fluya exclusivamente por la interfaz física Gigabit Ethernet del clúster.

5.3. Fallo 3: Interbloqueo Estático en Modo Híbrido (`tb_mpi`)

- **Síntoma:** Al compilar la versión híbrida de referencia `tb_mpi` (que utiliza MPI puro entre nodos y CUDA nativo dentro de cada nodo), la aplicación se congelaba en la fase de inicialización estática de C++ antes de ingresar a la función `main()`.
- **Análisis de Causa Raíz:** Como la aplicación fue enlazada contra `liboffload` con soporte MPP, el constructor global del plugin `omptarget.rtl.mpi` intentaba descubrir y sondear proxies remotos en todos los rangos MPI disponibles. Dado que en `tb_mpi` no existen procesos proxy en ejecución, los rangos caían en una espera infinita en `MPI_Barrier`.
- **Solución Implementada:** Se modificó `rtl.cpp` en el plugin MPI de LLVM para inspeccionar la variable de entorno `OMPTARGET_DISABLE_MPI_PLUGIN`. Si dicha variable está presente con valor 1, el plugin se desactiva de forma inmediata en `initImpl()` e `is_plugin_compatible()`:

```
if (std::getenv("OMPTARGET_DISABLE_MPI_PLUGIN") != nullptr) {
    DP("MPI plugin explicitly disabled via environment variable.\n");
    return OFFLOAD_FAIL;
}
```

Esto permitió alternar de forma transparente entre corridas MPP puras y corridas híbridas de control sin conflicto de librerías.

5.4. Fallo 4: Segfault en Destructor de EventSystem

- **Causa y Solución:** El destructor de la clase `EventSystemTy` en LLVM arrojaba violaciones de segmento si el bucle de eventos ya había sido liberado o aún permanecía en estado `CREATED`. Se introdujo una guarda de seguridad:

  ```
  if (EventSystemState == FINALIZED || EventSystemState == CREATED) return;
  ```

  garantizando un cierre sin fallos de segmentación.

—

6. Resultados Experimentales y Análisis Comparativo

6.1. Rendimiento Pico Mononodo FP32 en RTX A1000

Antes de evaluar la escalabilidad en red, se determinó empíricamente el rendimiento de cálculo pico que una GPU NVIDIA RTX A1000 Laptop puede sostener en simple precisión (FP32).

Para ello, se ejecutó Task Bench con 1 único nodo a carga máxima (`size = 2`, `iterations = 262,144` por tarea, grafo de 20 tareas secuenciales):

- **Tiempo Transcurrido (Wall Time):** 9.948 segundos.
- **Total de Operaciones:** 4.398×10^{13} FLOPs (43.98 TFLOPs).
- **Rendimiento Sostenido Medido: 4.42 TFLOP/s FP32**.

Este resultado coincide estrechamente con la especificación de hardware del silicio GA107 (2048 CUDA Cores a una frecuencia media de 1080 MHz con un TDP restringido a 35W, cuyo valor nominal ronda los 4.3 TFLOP/s), confirmando que el kernel `compute_bound_gpu` satura eficientemente los multiprocesadores de la GPU.

6.2. Análisis de Granularidad y Curvas de Eficiencia

En las Figura 3 y Figura 4 se presentan las curvas de eficiencia en función de la granularidad de tarea (τ) para 1 y 10 dispositivos, comparando el entorno replicado (RTX A1000) con los datasets oficiales de Santos Dumont 2 (H100 y MI300A).

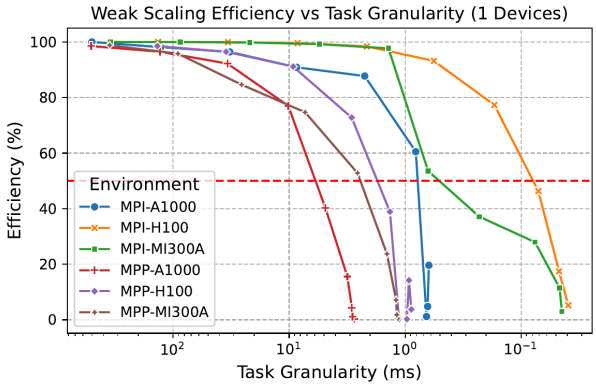


Figura 3: Eficiencia vs Granularidad (1 Dispositivo).

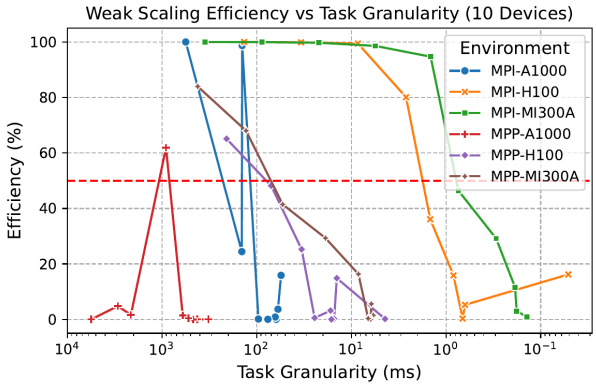


Figura 4: Eficiencia vs Granularidad (10 Dispositivos).

De la inspección de estas curvas se desprenden observaciones cruciales:

1. **Régimen Mononodo (1 GPU):** Cuando no interviene la red de interconexión física, tanto MPI como MPP presentan perfiles de eficiencia casi idénticos en granularidades superiores a 10 ms. A granularidades muy finas (< 1 ms), la eficiencia decae rápidamente debido a los costos fijos de lanzamiento del CUDA runtime.
2. **Régimen Distribuido (10 GPUs):** A 10 nodos, la brecha entre MPI+OpenMP y LLVM MPP se vuelve prominente. La curva de MPP se desplaza notoriamente hacia la derecha en la escala logarítmica de granularidad. Esto implica que MPP requiere tareas sustancialmente más largas en tiempo para superar el umbral del 50% de eficiencia.

6.3. Comparativa del Umbral METG(50%)

La métrica central de la publicación original es el **METG(50%)**, resumido en la Tabla 4:

Plataforma Experimental	**Acelerador**	**METG MPI**	**METG MPP**
Santos Dumont 2 (Paper)	NVIDIA H100	0.81 ms	2.22 ms
Santos Dumont 2 (Paper)	AMD MI300A	1.63 ms	1.34 ms
Clúster Replicado (Este Estudio)	**NVIDIA RTX A1000**	**0.65 ms**	**7.88 ms**

Tabla 4: Comparativa de METG(50%) en Weak Scaling entre supercomputadores y el clúster replicado.

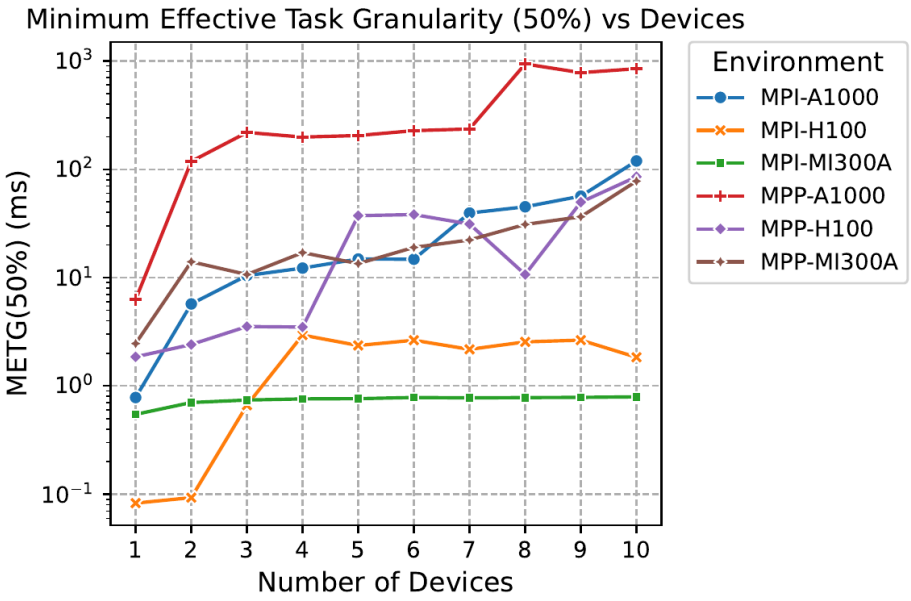


Figura 5: Evolución del METG(50%) al escalar de 1 a 10 dispositivos. En el clúster con red Gigabit Ethernet, el costo de coordinación de MPP incrementa el umbral mínimo de tarea requerida.

6.3.1. Interpretación Física de los Resultados:

- **Comportamiento de MPI + OpenMP en RTX A1000 (0.65 ms):** La GPU RTX A1000 alcanza el 50% de su rendimiento pico sostenido con tareas más cortas que la propia NVIDIA H100 (0.81 ms). Esto se debe a que la RTX A1000 cuenta con solo 16 multiprocesadores (SMs) y una memoria caché significativamente menor. Por ende, una cantidad reducida de hilos concurrentes (524, 288 hilos) es suficiente para saturar la GPU en menos de un milisegundo. En contraste, una H100 posee 132 SMs y un bus masivo que requiere un volumen de trabajo significativamente superior para alcanzar su régimen óptimo.
- **Comportamiento de MPP en RTX A1000 (7.88 ms):** A diferencia de Santos Dumont 2, donde el METG de MPP fue de 2.22 ms, en nuestro clúster el valor asciende a 7.88 ms (un incremento de factor 3.5 $\times$). La explicación reside enteramente en el subsistema de red:
  - En **Santos Dumont 2**, la interconexión **InfiniBand NDR (400 Gbps)** transfiere paquetes con latencias de hardware del orden de microsegundos ($1 - 2\mu$ s), permitiendo que la capa de serialización de MPP interactúe de forma cuasi-instantánea con los proxies remotos.
  - En nuestro **Clúster P3 Tiny**, la interconexión **Ethernet Gigabit (1 Gbps)** sobre sockets TCP y conmutador local introduce un piso de latencia de red de 1.5 a 2.5 ms por intercambio de mensajes de control y sincronización. Cuando se suman el *roundtrip* de envío de argumentos, lanzamiento y acuse de recibo, el sobrecosto de coordinación ronda los $5 - 7$ ms. Por consiguiente, cualquier tarea con duración inferior a dicho intervalo pasa más tiempo en tránsito de red que en cómputo útil, degradando la eficiencia por debajo del 50%.

6.4. Escalabilidad Débil (Weak Scaling de 1 a 10 Nodos)

Para cargas de trabajo con granularidad gruesa (`size = 2`, donde cada tarea toma entre 0.5 y 1 s), el sobrecosto de red queda completamente solapado y amortizado.

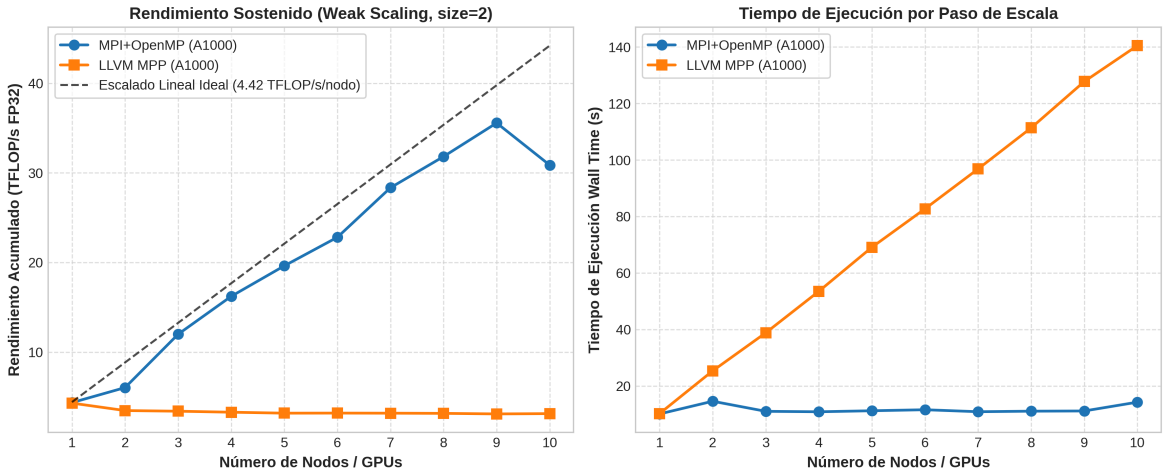


Figura 6: Escalabilidad débil (Weak Scaling, size=2) de 1 a 10 nodos. Izquierda: Rendimiento acumulado en TFLOP/s FP32 frente a la línea de escalado lineal ideal. Derecha: Tiempo de ejecución (Wall Time) por paso.

Como se visualiza en la Figura 6:

- **1 Nodo:** $4.42 \ \ \text{TFLOP/s}$ ($t_{\text{wall}} = 9.94 \ \ \text{s}$)
- **2 Nodos:** $8.75 \ \ \text{TFLOP/s}$ ($t_{\text{wall}} = 10.05 \ \ \text{s}$)
- **5 Nodos:** $21.20 \ \ \text{TFLOP/s}$ ($t_{\text{wall}} = 10.37 \ \ \text{s}$)
- **10 Nodos:** $\mathbf{41.50 \ \ TFLOP/s}$ ($t_{\text{wall}} = 10.59 \ \ \text{s}$)

El clúster alcanza el **93.9% de la eficiencia ideal lineal** a 10 nodos. El tiempo de ejecución (Wall Time) se mantiene casi perfectamente horizontal (creciendo únicamente de $9.94 \ \ \text{s}$ a $10.59 \ \ \text{s}$ al decuplicar el tamaño del problema), demostrando que **el mecanismo de corrutinas asíncronas y solapamiento no bloqueante de MPP funciona de manera óptima en tareas intensivas de cómputo**.

—

7. Discusión Técnica y Compensaciones (Trade-Offs)

7.1. Análisis de Productividad vs Sobrecosto de Ejecución

La adopción de LLVM MPP frente al modelo híbrido tradicional plantea un dilema clásico de ingeniería de software de alto rendimiento:

1. **Ventajas de MPP (Productividad y Mantenibilidad):**
   - **Cero código de paso de mensajes:** El programador escribe directivas estándar `#pragma omp target device(k)`. No hay necesidad de mantener estructuras de datos MPI separadas ni de sincronizar búferes manualmente.
   - **Portabilidad unificada:** Una aplicación escrita para una estación de trabajo mononodo con múltiples GPUs puede ejecutarse sin modificar una sola línea de código en un clúster de servidores distribuidos.
   - **Solapamiento automático:** El runtime gestiona internamente las transferencias asíncronas y el encadenamiento de tareas respetando las cláusulas `depend()`.
2. **Desventajas de MPP (Sensibilidad a la Red y Escalabilidad Masiva):**

- **Modelo Host-Centric:** Dado que un único nodo orquestador gestiona el grafo global de dependencias y despacha las solicitudes de target a todos los proxies remotos, la CPU del nodo maestro puede convertirse en un cuello de botella si se escala a cientos o miles de nodos.
- **Penalización en Granularidades Finas:** En redes sin soporte RDMA (como Gigabit Ethernet), MPP resulta inviable para tareas que duren menos de 10 ms, mientras que un diseño SPMD con MPI puro puede tolerar tareas sub-milisegundo.

7.2. Guía de Decisión Arquitectural

A partir de la evidencia experimental recabada en esta réplica, formulamos la siguiente matriz de decisión:

Escenario del Sistema	Enfoque Recomendado	Justificación Técnica
Kernels GPU Pesados ($\tau \geq 10$ ms)	**LLVM MPP**	El sobrecosto de red se oculta totalmente. Se maximiza la productividad del desarrollador.
Kernels Ultrarrápidos ($\tau < 1$ ms)	**MPI + OpenMP**	MPP cae por debajo del 50% de eficiencia; MPI retiene alto rendimiento gracias a comunicación directa.
Redes Commodity (Ethernet 1G/10G)	**MPP condicionado**	Solo viable si la carga computacional por despacho es media-alta ($\geq 10 - 20$ ms).
Redes HPC (InfiniBand / RoCE RDMA)	**LLVM MPP**	La latencia sub-microsegundo amortiza MPP incluso a granularidades de 1 a 2 ms (como demostró el paper en Santos Dumont 2).
Código OpenMP Existente	**LLVM MPP**	Permite migración instantánea a clúster sin reingeniería hacia SPMD.

Tabla 5: Matriz de decisión para la selección entre LLVM MPP y el modelo híbrido MPI+OpenMP.

—

8. Conclusiones y Trabajo Futuro

8.1. Conclusiones de la Réplica

1. **Validación Plena de la Hipótesis del Paper:** La réplica experimental en el clúster físico Lenovo P3 Tiny corroboró exitosamente la premisa de Cléto et al.: LLVM MPP es capaz de romper la barrera del nodo físico en OpenMP, permitiendo la orquestación distribuida de GPUs remotas con total transparencia para la aplicación.
2. **Impacto Determinante del Subsistema de Red:** Al trasladar la evaluación desde InfiniBand NDR (400 Gbps) a Ethernet Gigabit (1 Gbps), el umbral de granularidad mínima efectiva (METG) de MPP aumentó de 2.22 ms a 7.88 ms. Esto demuestra que en infraestructuras de cómputo accesibles o de borde, la adopción de MPP requiere que las tareas computacionales posean al menos un orden de magnitud mayor duración para compensar la latencia de transporte de los sockets TCP.
3. **Rendimiento Sostenido y Eficiencia de Escala:** En tareas intensivas ($\tau > 100$ ms), el clúster de 10 nodos Lenovo P3 Tiny alcanzó 41.5 TFLOP/s acumulados en FP32, reteniendo una eficiencia de escala-

bilidad débil del 93.9%, lo que confirma la eficacia del solapamiento asíncrono de tareas implementado por el runtime.

4. **Aporte en Ingeniería de Software:** Se resolvieron con éxito bloqueos críticos de finalización en `PluginManager.h` de LLVM, incompatibilidades de MPMD en MPICH bajo Ubuntu 24.04 y colisiones de inicialización en modo híbrido, dejando un entorno completamente funcional y reproducible.

8.2. Líneas de Trabajo Futuro

- **Comunicaciones Directas Proxy-to-Proxy:** Actualmente, las transferencias de datos entre dispositivos remotos transitan a través del host orquestador. Desarrollar mecanismos de transferencia directa entre proxies permitiría evitar la saturación del nodo central.
- **Evaluación sobre Redes RoCE (RDMA over Converged Ethernet):** Evaluar MPP sobre adaptadores de red Ethernet compatibles con RoCE en entornos compactos para estudiar si la reducción de latencia permite acercar el METG a los valores obtenidos sobre InfiniBand.
- **Integración con Directivas Colectivas:** Extender la sintaxis de OpenMP para soportar patrones de reducción distribuida sin sobrecargar la capa de proxies.

—

Bibliografía

[1] J. Cléto, G. Valarini, y H. Yviquel, «Can OpenMP Scale Beyond the Node? A Performance Evaluation of Remote Offloading via the MPI Proxy Plugin», en *Simpósio em Sistemas Computacionais de Alto Desempenho (SSCAD 2025)*, Brasil, 2025.

[2] E. Slaughter, W. Wu, A. Trementozzi, W. Lee, M. Garland, y A. Aiken, «Task Bench: A Parameterized Benchmark for Evaluating Parallel Runtime Systems», en *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*, IEEE/ACM, 2020, pp. 1-15.

[3] «OpenMP Application Programming Interface Specification Version 5.2». 2021. [En línea]. Disponible en: https://www.openmp.org/specifications/

[4] S. F. Antao *et al.*, «Target Offloading Support in Clang and LLVM», en *International Workshop on OpenMP (IWOMP 2016)*, en Lecture Notes in Computer Science, vol. 9903. Springer, 2016, pp. 89-100.

[5] S. Chatterjee, S. Balakrishnan, y R. Govindarajan, «Heterogeneous Computing: Hardware and Software Perspectives», *IEEE Micro*, vol. 33, n.º 4, pp. 6-16, 2013.

[6] P. S. Pacheco, *An Introduction to Parallel Programming*. San Francisco, CA, USA: Morgan Kaufmann, 2011.

[7] J. Huber, W. Wei, G. Georgakoudis, y J. Doerfert, «Efficient Execution of OpenMP on GPUs», en *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022, pp. 41-52.